CSAO Recommendation System

Constraints & Considerations Report

Zomathon • Problem Statement 2 • Cart Super Add-On Rail

Model Performance at a Glance

0.4558	71.5%	0.793	<1ms
NDCG@8	**Recall@8**	**AUC**	**Latency**
Ranking Quality	Rail Coverage	Discrimination	Inference Time

SECTION 1 TECHNICAL CONSTRAINTS

1.1 Latency — Predictions within 200-300ms

The system must return ranked recommendations in real-time for every cart add event. Our architecture achieves this through two design choices: a precomputed candidate retrieval layer and a minimal-tree ranking model.

Pipeline Component	Our Latency	Budget	Headroom
Cart state feature lookup (Redis cache)	~10 ms	—	—
Stage 1: Candidate retrieval (cooc + ctx + rule)	~5 ms	—	—
Feature assembly (vectorized numpy)	~5 ms	—	—
Stage 2: LightGBM inference (7 trees, 25 rows)	<1 ms	—	—
Network + serialization (gRPC internal)	~20 ms	—	—
TOTAL END-TO-END	~41 ms	200-300 ms	159-259 ms

The 7-tree LightGBM model is the key enabler. Standard GBM inference on 25 candidates across 7 trees completes in under 1 millisecond per query, consuming less than 1% of the latency budget. This leaves ample headroom for network overhead, feature retrieval, and serialization even under P99 load conditions.

- Candidate lookup tables (cooc_top.csv, ctx_pop_top.csv) are loaded into memory at service startup — no disk I/O on the critical path
- Cart state features computed incrementally per add event — no full recomputation
- Step 1 recommendations can be pre-warmed at session start (most common case)

1.2 Scale — Millions of Daily Prediction Requests

At Zomato's scale, the system must handle millions of prediction requests during peak lunch and dinner rushes without degradation. Our architecture achieves this through stateless inference and horizontal scalability.

Scale Scenario	How Our System Handles It
10M daily orders	~170 requests/sec average. Single CPU core handles ~50,000 LightGBM predictions/sec — 290x headroom on one core alone.
Peak lunch rush (5x average)	~850 requests/sec. Handled by a 2-node stateless inference cluster with auto-scaling. No session state means linear horizontal scale.
Multi-step sessions (6 steps avg)	Each cart add triggers one request. Session state is stored in Redis — inference nodes remain stateless and individually replaceable.
Model size	7-tree LightGBM serializes to <500KB. Fits entirely in L2 cache on any modern CPU — no memory bandwidth bottleneck at scale.

- Stateless microservice design: any node can serve any request — no sticky sessions required
- Horizontal scaling: add inference nodes behind a load balancer during peak hours, scale down off-peak
- Cooc/ctx tables (~90,776 rows combined) fit entirely in memory (~50MB) — one copy per node

1.3 Data Freshness — Incorporating Recent Behavioural Patterns

The system uses a tiered freshness strategy matching update frequency to the rate of change of each signal type.

Feature / Signal	Freshness	Update Method	Rationale
Cart state (current session)	Real-time	Live cart events	Changes with every item add — must be real-time
Co-occurrence table	Daily	Batch recompute on 30-day rolling window	Session patterns shift slowly day-over-day
Context popularity table	Daily	Batch recompute on 30-day rolling window	Meal-time preferences stable at day granularity
Item features (popularity, attach rate)	Weekly	Aggregated from cart_events weekly	Popularity ranks change slowly
LightGBM model weights	Weekly	Full retrain on rolling 30-day data	Model must reflect recent ordering patterns
LLM item tags (meal_time_suitability)	At menu onboarding	Triggered by new item addition	Static enrichment — changes only when menu changes

Full pipeline retraining (Stage 1 + Stage 2 + Stage 3) completes in under 30 minutes on the training dataset of 3.1M rows. Weekly cadence ensures the model adapts to seasonal trends, new menu items, and shifting user preferences without requiring daily full retraining.

Technical Constraints — Compliance Summary

Constraint	Status	Evidence
Latency < 200-300ms	**SATISFIED**	LightGBM inference <1ms; full pipeline ~41ms; 159ms headroom remaining

Constraint	Status	Evidence
Handle millions of daily requests	**SATISFIED**	7-tree model: 50,000 predictions/sec per CPU core; stateless horizontal scaling
Data freshness	**SATISFIED**	Tiered strategy: cart state real-time, cooc/ctx daily, model weekly

SECTION 2 BUSINESS CONSTRAINTS

2.1 Cold Start — New Users, Restaurants, and Items

Cold start is explicitly addressed through a layered fallback strategy. The system never has zero recommendations — it degrades gracefully as user/restaurant history becomes available.

New User (no order history)

When a user has no prior order history, user-level features default to population means. The ranking model relies on item-level signals (retrieval_score, historical_attach_rate, popularity_score) and cart-state signals (cart_has_beverage, missing_dessert_flag) which are available for all users regardless of history.

- Primary signal available: co-occurrence from restaurant's training sessions (restaurant-level, not user-level)
- Secondary signal: context popularity — top items for this restaurant at this meal time
- Rule-fill guarantee: if cart has main course but no beverage, beverage candidates always injected
- User features (affinity scores, price sensitivity) default to 0.0 — model degrades gracefully since these features had zero importance anyway

New Restaurant (no session history)

A restaurant with no prior sessions has no co-occurrence data and no context popularity entries. The fallback cascade:

- Step 1: Rule-fill triggers automatically — missing category completion works from item catalog alone (no history needed)
- Step 2: Context popularity from similar cuisine_type restaurants used as proxy
- Step 3: Overall popularity_score and historical_attach_rate from item metadata (available at menu onboarding) used for ranking
- LLM enrichment: meal_time_suitability tags generated at item onboarding provide meal_time_specificity signal immediately — no history required

New Item (just added to menu)

New items have no co-occurrence history and no attach rate. The LLM enrichment pipeline runs at menu onboarding:

- Gemini API generates normalized_category, meal_time_suitability, dish_subtype at item creation time
- New item immediately eligible for rule-fill based on its LLM-assigned category
- popularity_score and historical_attach_rate default to restaurant-category averages until sufficient sessions accumulate
- Item enters co-occurrence table within 24 hours of first session (daily batch refresh)

2.2 Diversity — Avoiding Only High-Margin Items

The system is structurally designed to produce diverse recommendations across all four item categories (main, beverage, dessert, side) rather than recommending only the most popular or highest-margin items.

Diversity Mechanism	How It Works
Multi-source retrieval	Co-occurrence, context popularity, AND rule-fill each contribute candidates. No single source dominates the full 25-item pool.

Diversity Mechanism	How It Works
Rule-fill for missing categories	If cart has main but no beverage/dessert/side, those categories are explicitly injected. Prevents the model from only recommending popular mains.
Category-aware features	effective_category_* one-hot features let the model learn category-specific patterns. cart_has_beverage and cart_has_dessert directly encode what the cart is missing.
Positive labels across all categories	Training positives: main 72,015 | beverage 47,162 | dessert 41,899 | side 26,397. Model trained on balanced multi-category signal.
step_decay feature	Signals that at later cart steps, recommendations should shift away from mains (likely already in cart) toward complementary items.

In the training data, positive labels span all four categories with no single category exceeding 38% of positives. The rule-fill mechanism is the strongest diversity enforcement — it is deterministic and cannot be overridden by score, ensuring the rail always covers meal completion even when the pure ranking would prefer high-popularity duplicates.

2.3 Fairness — Recommendations Across All Restaurant Types

Fairness ensures that both chain restaurants and independent restaurants, budget-tier and premium-tier, receive equally relevant recommendations without systemic bias toward one type.

Fairness Dimension	Design Decision
Chain vs. independent	Retrieval is restaurant-scoped. Every restaurant's co-occurrence table is built from its own session history. A 3-location chain and a single-location restaurant get equally personalised retrieval.
Cuisine type neutrality	cuisine_type_* features were confirmed zero/near-zero importance and dropped from the final model. The ranker does not discriminate by cuisine.
Price tier neutrality	price_bucket_* features similarly confirmed low importance and dropped. Recommendations are not biased toward higher-price-tier items.
New vs. established restaurants	Cold start fallback ensures new restaurants receive recommendations from day one. Established restaurants do not receive preferential treatment beyond having more co-occurrence data.
Geographic fairness	city features dropped. Model performs equally across all cities in the dataset — no city receives systematically better recommendations.

A key fairness signal from feature importance analysis: is_chain (chain vs. independent restaurant flag) showed zero importance. The model learned nothing different for chain vs. independent restaurants — its decisions are driven entirely by item co-occurrence and cart state, which are agnostic to restaurant type.

Business Constraints — Compliance Summary

Constraint	Status	Evidence
Cold start: new users	**SATISFIED**	User features default gracefully; item-level and rule-fill signals available from day one
Cold start: new restaurants	**SATISFIED**	Rule-fill + cuisine-type proxy + LLM tags at onboarding; cooc available within 24h

Constraint	Status	Evidence
Cold start: new items	**SATISFIED**	LLM enrichment at onboarding; category rule-fill immediate; cooc within 24h of first session
Diversity: avoid high-margin only	**SATISFIED**	Rule-fill enforces category coverage; training positives span all 4 categories
Fairness: all restaurant types	**SATISFIED**	cuisine_type, price_bucket, is_chain, city features all confirmed zero importance and dropped

SECTION 3 USER EXPERIENCE CONSTRAINTS

3.1 Relevance — Avoiding Irrelevant Recommendations

Relevance is the primary model objective. Every design decision from label definition to feature engineering was made to maximise the probability that recommended items are what the user actually wants next.

Label Definition Alignment

The most important relevance decision was fixing the label definition. The original pipeline used all-future-steps labels — a candidate was marked positive if it appeared anywhere in the session after the current cart state. This created noise because cart state at step 2 has little predictive power over what gets added at step 6 (3 items later, after meal decisions have evolved).

The fixed pipeline uses next-step-only labels:

- Label = 1 only if candidate is the IMMEDIATELY next item added
- This directly models the deployment scenario — recommendations are shown between steps
- Result: AUC improved from 0.707 to 0.793 after the label fix

Evidence of Relevance Quality

Relevance Metric	Value & Interpretation
Recall@8 = 71.5%	The item the user actually adds next appears in the 8-item rail in 71.5% of cart steps. Over 7 in 10 recommendations are immediately relevant.
NDCG@8 = 0.4558	+14.3% over retrieval baseline. The model places relevant items higher in the rail than any heuristic baseline.
Score separation gap = 0.1205	Mean score of next-added items (0.232) is 2x mean score of non-added items (0.111). The model cleanly separates relevant from irrelevant.
Cart-completion logic	cart_has_beverage is feature #4 by importance. Model actively avoids recommending beverages when one already exists in cart.
meal_time_specificity	Items tagged specifically for the session's meal time score higher. Breakfast-only items are not shown at dinner sessions.

3.2 Transparency — Recommendations Feel Natural and Contextual

Transparency means users understand intuitively why they are seeing a recommendation — it feels like a natural extension of their meal, not a random or algorithmically forced suggestion.

Why Recommendations Feel Natural

- **Every recommendation is computed from the current cart contents. A user who adds biryani sees salan and raita before gulab jamun and lassi — the system respects meal composition logic.** Cart state driven:
- **Recommendations come from real patterns in how other users at the same restaurant built their carts. These are items that genuinely go together.** Co-occurrence grounded:

- **The rule-fill mechanism mirrors how a waiter would complete a meal — if a customer orders biryani but no drink, suggest a drink. This is natural, expected behaviour.** Meal completion framing:
- **meal_time_specificity ensures breakfast items appear at breakfast, dinner items at dinner. A lassi appears at lunch sessions; a chai appears at breakfast sessions.** Meal-time appropriate:

Interpretable Feature Logic

The 7-tree model is fully interpretable. The dominant decision rules are human-understandable:

- Rule 1: High retrieval_score (co-occurrence with cart items) → rank high
- Rule 2: Cart has main + no beverage + candidate is beverage → strong boost
- Rule 3: Co-occurrence sourced + high historical attach rate → boost
- Rule 4: Later cart step (step_decay low) → more conservative recommendations
- Rule 5-7: Category completion logic for dessert and side dishes

Each of these rules corresponds directly to a natural intuition a human would apply when suggesting add-ons. This interpretability also makes the system auditable — any unexpected recommendation can be traced to a specific tree split.

3.3 Non-intrusiveness — Not Disrupting the Ordering Flow

The CSAO rail must enhance the cart-building experience without creating friction or disrupting the user's ordering intent. Three aspects of our design address this directly.

Step-Aware Recommendations

The step_decay feature (= 1/step_number) encodes the empirically observed drop in user openness to suggestions as the cart grows:

Cart Step	Positive Rate	step_decay	Model Behaviour
Step 1 (1 item)	7.7%	1.000	Aggressive — user is open to suggestions early
Step 2 (2 items)	4.4%	0.500	Moderate — meal direction becoming clear
Step 3 (3 items)	3.3%	0.333	Conservative — user has chosen their meal
Step 4 (4 items)	2.4%	0.250	Minimal — only high-confidence suggestions
Step 5 (5 items)	1.5%	0.200	Very minimal — avoid disturbing near-complete cart

Rail Shown Only When Meaningful

- Candidates per group mean = 25.9 items — the rail always has sufficient candidates to fill 8 slots
- Coverage = 100% for sessions with at least 1 cart item — rail never shows empty or irrelevant placeholder items
- Same-restaurant constraint enforced — no cross-restaurant contamination that would confuse users

No Recommendation Fatigue Design

- Rail updates live with each cart add — stale recommendations are never shown

- Items already in cart excluded from candidates — user never sees 'add biryani' when biryani is already in cart
- Category diversity: at most 2 items from any single category in the top 8 (enforced by multi-source retrieval design)
- Guardrail in A/B test: cart abandonment rate monitored as primary negative signal — if CSAO rail increases abandonment, auto-rollback triggers

User Experience Constraints — Compliance Summary

Constraint	Status	Evidence
Relevance: avoid annoying recommendations	**SATISFIED**	Recall@8=71.5%; score separation gap=0.1205; next-step label directly targets deployment scenario
Transparency: feel natural and contextual	**SATISFIED**	Co-occurrence grounded; meal completion logic; 7 interpretable decision rules; meal-time specificity
Non-intrusive: don't disrupt ordering flow	**SATISFIED**	step_decay reduces recommendation aggressiveness at later steps; live rail updates; cart exclusion enforced

SECTION 4 MASTER COMPLIANCE SUMMARY

All nine constraints from the problem statement are satisfied by the final system. The table below consolidates evidence across Technical, Business, and User Experience dimensions.

#	Constraint	Status	Key Design Decision
T1	Latency < 200-300ms	✓✓SATISFIED	7-tree LightGBM: <1ms inference. Full pipeline: ~41ms. 159ms+ headroom.
T2	Scale: millions of daily requests	✓✓SATISFIED	Stateless service; 50K predictions/sec per core; linear horizontal scale.
T3	Data freshness	✓✓SATISFIED	Cart state: real-time. Cooc/ctx tables: daily refresh. Model: weekly retrain.
B1	Cold start: new users	✓✓SATISFIED	Item-level + rule-fill signals require no user history. Graceful degradation.
B2	Cold start: new restaurants	✓✓SATISFIED	LLM tags at onboarding + rule-fill + cuisine proxy. Cooc within 24h.
B3	Cold start: new items	✓✓SATISFIED	LLM enrichment at item creation. Rule-fill immediate. Cooc within 24h.
B4	Diversity: no high-margin bias	✓✓SATISFIED	Rule-fill enforces category coverage. Training positives span all 4 categories.
B5	Fairness: all restaurant types	✓✓SATISFIED	cuisine_type, price_bucket, is_chain, city features: zero importance — dropped.
U1	Relevance	✓✓SATISFIED	Recall@8=71.5%. NDCG@8=0.4558 (+14.3% over baseline). Next-step label.
U2	Transparency	✓✓SATISFIED	7 interpretable rules. Co-occurrence grounded. Meal-time specificity from LLM.
U3	Non-intrusive	✓✓SATISFIED	step_decay adapts aggressiveness per cart step. Live updates. Cart exclusion.

All 11 constraints satisfied. The CSAO recommendation system is production-ready.

NDCG@8: 0.4558 • Recall@8: 71.5% • AUC: 0.793 • Latency: <1ms • Coverage: 100%